HanginOut

Development Requirements Document (DRD)

Student-focused affordable hangout discovery and collaborative group matching platform

Project tagline: Making memories shouldn't cost ₹1000.

Document purpose:w This document defines the product requirements, features, database, APIs, architecture, UI flow, security, development approach, and project scope for building HanginOut.

1. Project Overview

HanginOut is a student-focused web platform that helps users discover free and low-cost hangout experiences based on budget, interests, group type, duration, and location. It uses a Tinder/Hinge-style swipe interface so users can quickly like or reject activities. For groups, each member can independently swipe and the system identifies activities liked by all or most members.

Target Users

- School and college students.
- Budget-conscious groups of friends.
- Families, and solo users looking for affordable activities.
- Users who want a faster way to decide what to do together.

Problem Statement

School and college students often want to spend quality time with friends but face limited budgets and difficulty deciding what activities they can do together. Students commonly rely on friends, social media, or scattered searches. The problem becomes harder when group members have different interests, budgets, and preferences.

Product Objective

Make discovering and planning student-friendly hangouts affordable, simple, interactive, and collaborative while helping users discover new experiences without requiring a large budget.

2. Project Goals

- Provide secure registration and login with JWT authentication.
- Collect user preferences such as budget, interests, group type, duration, and city.
- Display affordable hangout activities using swipe cards.
- Allow users to like, reject, save, and view activity details.
- Provide rule-based personalized recommendations.
- Allow users to create/join groups using an invite code.

- Allow each group member to swipe independently.
- Find activities liked by all or most group members.
- Allow the group to create a simple hangout plan.

- Provide an admin interface for managing activities, spots, categories, users, and feedback.
- Use dummy/manual activity and spot data for now.

3. Technology Stack

Layer	Technology
Frontend	React + Vite + JavaScript + React Router + Axios
UI	Tailwind CSS + Framer Motion
Backend	Java + Spring Boot + Spring Web
Security	Spring Security + JWT + BCrypt
Persistence	Spring Data JPA + Hibernate
Validation	Jakarta Bean Validation
Database	PostgreSQL
Images	Cloudinary or static image URLs initially
API Documentation	Swagger / OpenAPI
Version Control	Git + GitHub
Deployment	Vercel (frontend), Render (backend), Neon PostgreSQL (database)

4. System Architecture

The system follows a **modular client-server architecture**.

React communicates with Spring Boot through REST APIs. The backend is organized as a **modular monolith** with clear domain boundaries.

Controllers handle HTTP requests, services contain business logic, repositories handle database access, and Spring Security manages authentication and authorization. PostgreSQL is accessed through Spring Data JPA/Hibernate.

External services such as image hosting, location services, and Places APIs can be integrated later without tightly coupling them to the core application.

Architectural Principles

- Each major domain should be implemented as a separate module/component, such as:
    - Authentication
    - Users

    - Preferences
    - Activities
    - Recommendations
    - Swipes
    - Saved Activities
    - Groups
    - Hangout Plans
    - Feedback
    - Administration
- Modules should communicate through well-defined service interfaces and should avoid unnecessary direct dependencies on unrelated modules.
- Business logic should remain in service/domain layers rather than controllers or repositories.
- Controllers should expose DTOs rather than directly exposing JPA entities.
- Request and response DTOs should define stable API contracts between the frontend and backend.
- Entity relationships should not be directly serialized into API responses to avoid circular references and unnecessary data exposure.
- The architecture should allow individual modules to be modified or extended without requiring major changes to unrelated modules.
- The application should remain a **modular monolith**, not a microservices architecture.
- All modules initially run within the same Spring Boot application.
- The modular boundaries should allow future extraction of individual modules into separate services if the project grows significantly, although such extraction is outside the current scope.

Request Flow

React Frontend

↓

REST API

↓

Controller

↓

Service

↓

Repository

↓

PostgreSQL

Additional architecture requirements:

- JWT is sent in the Authorization header for protected endpoints.
- Recommendation logic is isolated in its own service so it can later be replaced by an ML-based system.
- Activity and spot data is manually seeded initially.
- A Places API can be integrated later without redesigning the core activity model.
- Frontend and backend are developed independently and communicate over HTTP.
- Frontend and backend do not need to run on the same laptop.

5. Functional Requirements

5.1 User Authentication

- POST /api/auth/register
- POST /api/auth/login
- Passwords must be securely hashed using BCrypt.
- Successful login returns a JWT.
- Protected endpoints require a valid JWT.
- User email must be unique.
- Authentication failures should return appropriate HTTP status codes.

5.2 User Profile

- Users can view and update their profile.
- Profile information includes:
  - Name
  - Age
  - City
  - Other basic profile information where applicable.
- Endpoints:
  - GET /api/users/me
  - PUT /api/users/me

5.3 Onboarding and Preferences

Users should complete an onboarding flow after registration.

Budget

- Free
- Under ₹100
- Under ₹300
- Under ₹500
- ₹500+

Group Type

- Friends
- Solo

Interests

Examples:

- Food
- Adventure
- Outdoors
- Games

- Movies
- Creative
- Exploring
- Sports
- Chill

Duration

- Less than 1 hour
- 1–2 hours
- 2–4 hours
- 4+ hours

Preferences should be stored and used for filtering and recommendation scoring.

5.4 Activity Discovery

- Display activities using swipeable cards.
- The initial card should prioritize the primary activity image.
- The collapsed card should show only subtle essential information:
    - Activity name
    - Estimated cost
    - Category
    - City/location
- Detailed information should not clutter the initial swipe experience.
- Users can filter activities by:
    - Bud
    - get
    - Category
    - Duration
    - Group type
    - City/location
    - Distance where location coordinates are available.
- Activity discovery APIs should support pagination.

5.5 Swipe System

- Left swipe = reject.
- Right swipe = like.
- Provide visible button alternatives for users who do not use gestures.
- Store every swipe with:
    - User
    - Activity
    - Action
    - Timestamp
- Prevent duplicate active swipe records for the same user and activity.

5.6 Expandable Activity Details

The activity card should have two primary states.

<u>Collapsed / Swipe State</u>

The default state should provide a visually focused, Hinge-inspired card.

It should primarily display:

- Large activity image
- Activity name

- Estimated cost
- Category
- Location/city
- Like/reject controls

Detailed information should remain hidden to keep the swipe interface visually clean.

Expanded / Detail State

Users should be able to vertically drag or scroll the card upward to transition into a full-screen activity detail view.

The expanded state should display:

- Additional activity images
- Full description
- Estimated cost
- Duration
- Category
- Best time
- Group suitability
- Indoor/outdoor information
- Required items
- Instructions
- Associated spots/places
- Location information

Users should be able to collapse the expanded view and return to the swipe state.

Gesture Requirements

- Horizontal gestures should control like/reject only when the card is in the collapsed swipe state.
- Vertical gestures should control card expansion/collapse.
- Vertical scrolling in the expanded state must not accidentally trigger like/reject.
- Like/reject buttons must remain available in the collapsed state.
- The transition between collapsed and expanded states should be smooth and responsive.
- The interaction must work with mouse and touch input.

5.7 Categories

- Categories are stored in a separate table.
- Categories can be managed by administrators.
- Activities can belong to a category.

5.8 Budget System

• Activities and spots contain minimum and maximum estimated costs.

• Budget filters compare user budget against activity cost.

• Costs are estimates and are not treated as booking prices.

5.9 Recommendation System

• The initial system uses a rule-based scoring algorithm, not ML.

• ML recommendation must NOT be implemented during the initial development stage. The initial implementation should only create the recommendation architecture/interface required to support the current rule-based system and future ML integration.

• Example weights: budget +30, interest/category +25, group +20, duration +15, location +10.

• Previous likes/dislikes can adjust the score.

• Recommendation code must be isolated in a dedicated service.
• Controllers should not contain recommendation logic.
• The recommendation service should expose a stable interface so the underlying algorithm can be replaced later.

5.10 Swipe History

• Store user/activity/action/timestamp.

• swipe History supports recommendation logic and analytics.

5.11 Saved Activities / My Hangouts

• Users can save an activity.

• Users can view saved activities.

• Users can remove saved activities.

• Saved activities should be unique per user/activity combination.

5.12 Group Hangout

- Users can create a group.
- A group has:
  - Name
  - Creator
  - Generated invite code
  - Status
  - Creation timestamp
- Example invite code: HNG4821
- The creator automatically becomes a group member.
- Users can join using an invite code.
- Groups can have active or closed status.

5.13 Group Swiping

• Each group member can independently like/reject an activity.

• Store group, user, activity, action, and timestamp.

• Prevent duplicate member/activity swipe records.

5.14 Group Matching

- Find activities liked by all group members.
- If no unanimous match exists, identify activities liked by the highest percentage of members.
- Return:
  - Activity
  - Match percentage
  - Member responses

5.15 Match Result

• Display a prominent 'IT’S A MATCH!' state for a unanimous group match.

• Show a small celebratory confetti animation.

• Also show high-percentage alternatives when there is no unanimous match.

5.16 Hangout Planning

Groups can create a simple hangout plan containing:

- Group
- Activity
- Optional spot
- Date
- Time
- Status

Users can:

- Create plans
- View plans
- Update plans
- Delete plans

5.17 Spots / Places

Activities and physical spots should remain separate concepts.

Spots should initially be manually seeded.

A spot should store:

- Name
- Description
- Address
- City
- Latitude
- Longitude
- Minimum cost
- Maximum cost
- Opening hours
- Active status
- Images

Later, Google or another Places API can be integrated without redesigning the core activity model.

5.18 Feedback

• Users can submit 1–5 rating, enjoyed yes/no, and optional comment.

• Feedback may optionally be associated with a group.

• Admin can view feedback.

5.19 Admin Dashboard

Administrators can:

- Create, update, and delete activities.
- Create, update, and delete categories.
- Create, update, and delete spots.
- View/manage users.
- View feedback.
- View basic analytics.

5.20 Analytics

Track:

- Most liked activities
- Most disliked activities
- Matched activities
- Popular categories
- Popular budget ranges
- Number of swipes
- Number of groups
- Number of matches

6. REST API Requirements

6.1 Authentication

- POST /api/auth/register
- POST /api/auth/login

6.2 User

- GET /api/users/me
- PUT /api/users/me

6.3 Preferences

- GET /api/preferences
- PUT /api/preferences

6.4 Activities

- GET /api/activities
- GET /api/activities/{id}
- GET /api/activities/recommended

Supported filters may include:

- maxBudget

- category
- city
- duration
- distance

Activity discovery should support pagination, for example:

GET /api/activities?page=0&size=10

Distance filtering should use latitude/longitude when location information is available. It should not require a third-party Places API during the initial implementation.

6.5 Swipes

- POST /api/swipes
- GET /api/swipes
- GET /api/swipes/liked

6.6 Saved Activities

- GET /api/saved
- POST /api/saved/{activityId}
- DELETE /api/saved/{activityId}

6.7 Groups

- POST /api/groups
- GET /api/groups/{id}
- POST /api/groups/join
- GET /api/groups/{id}/members

6.8 Group Swipes

- POST /api/groups/{id}/swipes
- GET /api/groups/{id}/matches

6.9 Plans

- POST /api/groups/{id}/plans
- GET /api/groups/{id}/plans
- PUT /api/plans/{id}
- DELETE /api/plans/{id}

6.10 Feedback

- POST /api/feedback
- GET /api/activities/{id}/feedback

API Standards

All APIs should:

- Use JSON.

- Use appropriate HTTP status codes.
- Use request and response DTOs.
- Use consistent error responses.
- Validate incoming requests.
- Enforce authentication where required.
- Enforce role-based authorization for admin endpoints.
- Be documented using Swagger/OpenAPI.

7. Database Design

Table	Important columns / purpose
users	id PK, name, email UNIQUE, password_hash, age, city, role, created_at, updated_at
categories	id PK, name UNIQUE, description
activities	id PK, title, description, min_cost, max_cost, duration_minutes, best_time, location_type, group_type, image_url, city, is_active, created_at, updated_at, category_id FK
activity_spots	activity_id FK, spot_id FK; composite PK
spots	id PK, name, description, address, city, latitude, longitude, min_cost, max_cost, image_url, opening_hours, is_active, timestamps
user_preferences	id PK, user_id FK UNIQUE, max_budget, preferred_group_type, preferred_duration, city
user_interests	user_id FK, category_id FK; composite PK
swipes	id PK, user_id FK, activity_id FK, action, created_at; UNIQUE(user_id, activity_id)
saved_activities	id PK, user_id FK, activity_id FK, created_at; UNIQUE(user_id, activity_id)
hangout_groups	id PK, name, invite_code UNIQUE, creator_id FK, status, created_at
group_members	id PK, group_id FK, user_id FK, joined_at; UNIQUE(group_id, user_id)
group_swipes	id PK, group_id FK, user_id FK, activity_id FK, action, created_at; UNIQUE(group_id,user_id,activity_id)
hangout_plans	id PK, group_id FK, activity_id FK, spot_id FK nullable, planned_date, planned_time, created_at, status
feedback	id PK, user_id FK, activity_id FK, group_id FK nullable, rating, enjoyed, comment, created_at

Image Design

Activities and spots may contain multiple images.

Images should therefore not be stored as a single image_url field in the activity/spot tables.

Instead:

- activity_images stores multiple images for an activity.
- spot_images stores multiple images for a physical spot.
- display_order determines the order in which images appear in the frontend gallery.

This supports the expandable activity-card experience.

8. Database Relationships

- User has one UserPreference record.
- Users and Categories have a many-to-many relationship through UserInterests.
- One Category has many Activities.
- One Activity can have many ActivityImages.
- One Spot can have many SpotImages.
- Activities and Spots have a many-to-many relationship through ActivitySpots.
- Users and Activities have a many-to-many interaction through Swipes.
- Users and Activities have a many-to-many saved relationship through SavedActivities.
- One User can create many HangoutGroups.
- Groups and Users have a many-to-many relationship through GroupMembers.
- Group members and Activities interact through GroupSwipes.
- A Group can have multiple HangoutPlans.
- An Activity can have multiple Feedback records.
- Feedback may optionally reference a Group.

9. Frontend Requirements

The frontend should include:

- Landing page
- Register
- Login
- Onboarding/preferences
- Discover/swipe screen
- Expandable activity cards
- Saved Activities / My Hangouts
- Create Group
- Join Group

- Group Dashboard
- Group Swipe
- Match Results
- Hangout Plan
- Profile
- Admin Dashboard

The activity detail experience should primarily be implemented through the expandable activity card rather than requiring a separate page for every activity.

10. UI/UX Requirements

General Design

- Youthful, clean, responsive design.
- HanginOut should have its own visual identity while taking inspiration from Hinge's card-based interaction model.
- Design should work on desktop, tablet, and mobile browsers.
- Use clear visual hierarchy for cost, category, duration, and location.
- Show loading, empty, success, and error states.
- Use smooth but lightweight animations.
- Show a small celebratory confetti effect on a group match.

Activity Card

- The activity discovery interface should follow a **Hinge-inspired expandable card interaction**.

Collapsed Swipe State

The default card should:

- Prioritize a large primary image.
- Show minimal information.
- Display activity name.
- Display estimated cost.
- Display category.
- Display location/city.
- Provide like/reject actions.

Detailed information should not clutter the initial swipe experience.

Expanded Detail State

Users should be able to vertically drag or scroll the card upward.

The card should then transition into a full-screen detail state containing:

- Additional images.

- Image gallery/carousel.
- Full description.
- Estimated cost.
- Duration.
- Category.
- Best time.
- Group suitability.
- Indoor/outdoor information.
- Required items.
- Instructions.
- Associated spots.
- Location information.

Users should be able to collapse the expanded card and return to the swipe state.

Gesture Behavior

- Horizontal gesture → like/reject.
- Vertical gesture → expand/collapse.
- Horizontal gestures should not trigger like/reject while the user is vertically scrolling the expanded content.
- Visible buttons must provide an alternative to gestures.
- The interaction must support mouse and touch.
- The transition between states should be smooth and responsive.

Accessibility

- Like/reject buttons must always be available as an alternative to gestures.
- Important information should not rely exclusively on animation or color.
- Interactive controls should have clear labels.
- Text should remain readable across desktop and mobile layouts.

Frontend Development

- The frontend can initially use mock JSON data.
- API integration should be added once backend contracts are stable.
- Axios should be used for backend communication.
- React Router should handle application navigation

11. Security Requirements

- ☐ Hash passwords with BCrypt.
- ☐ Never store plaintext passwords.
- ☐ Use JWT for authenticated API access.
- ☐ Do not expose password hashes in API responses.
- ☐ Validate incoming request data using Jakarta Bean Validation.
- ☐ Sanitize/validate data where appropriate.
- ☐ Use role-based authorization for admin endpoints.
- ☐ Configure CORS for the deployed frontend origin rather than allowing unrestricted production access.
- ☐ Use environment variables for:
  - JWT signing keys
  - Database credentials
  - External API keys
  - Other secrets
- ☐ Do not commit secrets to GitHub.

12. Error Handling

Use appropriate HTTP status codes including:

- 200 OK
- 201 Created
- 204 No Content
- 400 Bad Request
- 401 Unauthorized
- 403 Forbidden
- 404 Not Found
- 409 Conflict

Return a consistent JSON error structure containing information such as:

```
{
 "timestamp": "2026-10-07T12:00:00",
 "status": 400,
 "message": "Invalid request",
 "path": "/api/groups/join"
}
```

Handle the following cases cleanly:

- Duplicate email
- Invalid invite code
- Unauthorized access
- Forbidden admin access
- Invalid swipe

- Duplicate swipe
- Missing activity
- Missing group
- Invalid request data
- Duplicate saved activity

13. Seed / Dummy Data

- ☐ Create at least 20–30 activities across multiple categories.
- ☐ Create multiple spots across the initial target city.
- ☐ Use varied budgets:
  - Free
  - Under ₹100
  - ₹100–₹300
  - ₹300–₹500
  - Higher-cost examples
- ☐ Include different durations.
- ☐ Include different group types.
- ☐ Include indoor/outdoor activities.
- ☐ Include multiple categories.
- ☐ Use placeholder/static image URLs initially.
- ☐ Activities should have multiple images where possible to demonstrate the expanded gallery.
- ☐ Seed categories before activities.
- ☐ Ensure all foreign keys are valid.
- ☐ Seed realistic but manually maintained data initially.

14. Frontend–Backend Coordination

- ☐ Frontend and backend do not need to run on the same laptop.
- ☐ Both teams should agree on request/response contracts before implementation.
- ☐ Swagger/OpenAPI is the primary API reference.
- ☐ Frontend can use mock JSON while backend APIs are being built.
- ☐ Use GitHub feature branches and pull requests.
- ☐ Backend should provide:
  - Endpoint names
  - HTTP methods
  - Request schemas
  - Response schemas
  - Authentication requirements

- Error formats

☐ Test APIs with Postman before frontend integration.

☐ API changes should be documented before frontend integration.

15. GitHub Repository Structure

```
hanginout/
├── backend/
├── frontend/
├── docs/
│   ├── DRD.pdf
│   ├── api-specification/
│   └── database/
└── README.md
```

Backend Feature Structure

The backend should follow a **feature-based modular structure** with layers inside each feature.

Example:

```
backend/
└── src/main/java/com/hanginout/
    ├── auth/
    │   ├── controller/
    │   ├── service/
    │   ├── repository/
    │   ├── entity/
    │   └── dto/
    │
    ├── user/
    ├── preference/
    ├── activity/
    ├── recommendation/
    ├── swipe/
    ├── saved/
    ├── group/
    ├── plan/
    ├── feedback/
    ├── admin/
    │
    └── common/
        ├── exception/
```

```
├── response/
├── security/
└── config/
```

Each module should own its relevant controllers, services, repositories, entities, and DTOs.

Example branches:

- backend/auth
- backend/activities
- backend/groups
- backend/recommendation
- frontend/discover
- frontend/group
- frontend/profile

16. Project Scope

- The initial implementation is a web application.
- Native mobile development is outside the current scope.
- AI chatbot is a future extension.
- ML recommendation is a future extension.
- Microservices are outside the current scope.
- HanginOut should initially be implemented as a modular monolith with clear module boundaries.
- Complex Maps integration is deferred.
- Manually seeded spots are used initially.
- Payments are outside the current scope.
- Real-time chat is outside the current scope.
- Public social feed is outside the current scope.
- Business accounts are outside the current scope.
- Booking marketplace functionality is outside the current scope.
- Automatic price scraping is outside the current scope.
- Push notifications are outside the current scope.

17. Recommendation System Evolution

Phase 1 — Rule-Based

Score activities using:

- Budget
- Interests
- Group type
- Duration
- Location
- Interaction history

Phase 2 — Data Collection

Use swipe history as implicit preference data.

Phase 3 — ML

Train a model to estimate the probability that a user will like an activity.

Phase 4 — Advanced Personalization

Combine:

- User history
- Activity metadata
- Location
- Group behavior
- Contextual signals
- Potentially weather and time-based information

The recommendation interface must remain separate from controllers and repositories so the recommendation algorithm can be replaced without rewriting unrelated application modules.

18. Edge cases and exception handling

The application must handle unexpected user actions, invalid inputs, missing data, authorization failures, network errors, and unusual business scenarios gracefully.

The system must not crash, expose sensitive information, create inconsistent database records, or leave the user uncertain about whether an action succeeded.

18.1 Authentication and User Accounts

- **Duplicate registration:** If a user registers with an email that already exists, reject the request with a clear message.
- **Invalid credentials:** If the email or password is incorrect, return a generic authentication error without revealing which field was incorrect.
- **Invalid or expired JWT:** Reject protected requests and require the user to log in again when appropriate.
- **Missing authentication:** Reject requests to protected endpoints when the JWT is missing.
- **Invalid registration data:** Reject missing fields, malformed emails, blank names, and passwords that do not meet the configured requirements.
- **Repeated form submission:** Prevent accidental duplicate registrations or duplicate requests caused by repeated clicks.

- **Unauthorized profile access:** Users must not be able to access or modify another user's private profile data by changing an ID in a request.
- **Inactive or deleted account:** Prevent an inactive account from accessing protected functionality.

18.2 Onboarding and Preferences

- **Incomplete onboarding:** Users who have not completed required preferences should be directed to complete onboarding before accessing personalized discovery, if onboarding is mandatory.
- **Missing optional preferences:** The recommendation system should still function when optional preferences are not provided.
- **Invalid budget or duration:** Reject unsupported values rather than silently accepting them.
- **Changing preferences:** Updating preferences should affect future recommendations without deleting existing swipe or saved-activity history.
- **Empty interests:** If a user selects no interests, provide general activities instead of returning an error.

18.3 Activity Discovery and Recommendations

- **No matching activities:** Display a friendly empty state and allow users to change filters or reset them.
- **No activities in a city:** Show a message explaining that activities are not currently available for the selected city.
- **No activities within budget:** Suggest relaxing the budget filter or exploring free alternatives.
- **Missing activity image:** Display a default placeholder image without breaking the card layout.
- **Missing optional activity details:** Hide unavailable fields or show an appropriate fallback instead of displaying errors or undefined values.
- **Inactive activity:** Do not show inactive activities in normal discovery or recommendations.
- **Activity deleted after loading:** Handle a missing activity gracefully when the user attempts to like, save, or open it.
- **Invalid filters:** Reject invalid filter values with a useful error response.
- **Pagination beyond available results:** Return an empty result list with valid pagination metadata instead of an application error.
- **Duplicate recommendations:** Do not repeatedly recommend the same activity in the same discovery session unless the product explicitly allows it.
- **Previously swiped activities:** Exclude previously swiped activities from the default discovery feed, unless the user is viewing swipe history or another explicitly defined screen.
- **Equal recommendation scores:** Use a consistent secondary ordering rule so results do not change unpredictably between requests.
- **Insufficient recommendation data:** Fall back to rule-based scoring using the preferences and activity metadata that are available.
- **ML unavailable in the future:** Retain a working rule-based recommendation fallback if an ML model is introduced later.

18.4 Swipe and Saved Activities

- **Duplicate swipe:** Prevent duplicate active swipe records for the same user and activity.
- **Repeated swipe request:** Handle repeated requests safely and avoid creating duplicate records.
- **Invalid swipe action:** Accept only the supported actions, such as LIKE and REJECT.
- **Unauthorized swipe:** Prevent users from submitting swipes on behalf of another user.
- **Swipe on inactive activity:** Reject the request if the activity is no longer available for discovery.
- **Swipe request fails:** Show an error and allow the user to retry without accidentally submitting the action twice.
- **Save the same activity twice:** Do not create duplicate saved-activity records.

- **Remove an unsaved activity:** Return an appropriate response without causing an application error.
- **Saved activity becomes inactive:** Handle it gracefully in the saved list and prevent it from appearing in new discovery results.
- **Like/reject button clicked repeatedly:** Disable or debounce the relevant control while a request is being processed, while also enforcing database constraints on the backend.

18.5 Expandable Activity Card and UI Gestures

- **Missing primary image:** Show a placeholder image and preserve the card's layout.
- **Single-image activity:** Display the activity normally without requiring a gallery.
- **No images available:** Show a fallback visual rather than an empty or broken image container.
- **Gesture conflict:** Vertical expansion or scrolling must not trigger a horizontal like/reject action.
- **Gesture at the wrong state:** Horizontal swipes must not trigger like/reject while the card is in expanded detail mode.
- **Accidental gesture:** Provide visible like/reject buttons as an alternative to gestures.
- **Rapid gestures:** Prevent multiple activities from being accidentally liked or rejected while the previous action is still processing.
- **Failed image loading:** Preserve the rest of the card and allow the user to continue using the application.
- **Responsive layout:** Ensure the card and expanded view remain usable on small screens, tablets, and desktop browsers.

18.6 Groups and Invitations

- **Invalid invite code:** Display a clear error when a code does not correspond to an active group.
- **Expired, closed, or unavailable group:** Prevent users from joining a group that is no longer accepting members.
- **Joining the same group twice:** Do not create duplicate membership records.
- **Creator membership:** Ensure the group creator is automatically added as a member exactly once.
- **Group with one member:** Allow the group to exist and behave correctly even if only one member participates.
- **Group with no members:** Prevent invalid group states; group creation must add the creator transactionally.
- **Unauthorized group access:** Only group members and authorized administrators should access private group details, swipes, matches, and plans.
- **Creator or member leaves:** Handle membership changes consistently and define how group ownership is transferred or how the group is closed.
- **Group closes during a request:** Reject new joins, swipes, or plans when the operation is not allowed for a closed group.
- **Invite code collision:** Generate a new code if the generated invite code already exists.
- **Concurrent joins:** Prevent duplicate membership records when multiple requests attempt to add the same user simultaneously.

18.7 Group Swiping and Matching

- **Member has not swiped:** Do not treat a missing response as a rejection. Distinguish between pending and rejected responses.
- **No unanimous matches:** Display the highest-percentage alternatives when available.
- **No positive matches:** Show a friendly message and allow members to continue discovering activities.
- **Tie between activities:** Return tied activities using a consistent secondary ordering rule.
- **Member joins after swiping begins:** Define whether the new member participates in the current

matching round or only future rounds. Apply the same rule consistently.

- **Member leaves during matching:** Recalculate match percentages using the active membership rules.
- **Member submits duplicate group swipe:** Prevent duplicate group/user/activity records.
- **Non-member submits a group swipe:** Reject the request.
- **Unauthorized match access:** Prevent users outside the group from viewing its matching results.
- **Changing a previous swipe:** If swipe changes are supported, update the existing record rather than creating a duplicate.
- **Concurrent group swipes:** Ensure simultaneous submissions do not produce inconsistent matching results.
- **Match percentage:** Calculate percentages using the explicitly defined eligible group membership and valid responses. Do not count pending responses as rejections.

18.8 Hangout Planning

- **Missing required fields:** Reject a plan without required information such as group, activity, date, or time.
- **Invalid date or time:** Validate date and time formats and reject dates in the past if the product's planning rules prohibit them.
- **Unavailable activity:** Prevent a plan from referencing a deleted or inactive activity.
- **Invalid spot:** Ensure the selected spot exists and is associated with the activity where that relationship is required.
- **Unauthorized plan modification:** Only authorized group members should create, view, or modify group plans, according to the defined permissions.
- **Duplicate plan submission:** Prevent accidental duplicate plans caused by repeated requests.
- **Group closed or membership changed:** Apply the defined group and membership permissions before allowing plan changes.
- **Plan deletion:** Return a clear success response and handle attempts to delete nonexistent plans appropriately.

18.9 Spots, Images, and External Services

- **Missing coordinates:** Allow activities or spots without coordinates, but disable distance-based filtering for those records.
- **Invalid coordinates:** Reject latitude or longitude outside valid geographic ranges.
- **Missing opening hours:** Display that opening hours are unavailable instead of assuming the spot is open.
- **Invalid image URL:** Use a fallback image when an image cannot be loaded.
- **External service unavailable:** If image hosting or a future Places API fails, preserve core discovery functionality where possible.
- **Incomplete external data:** Validate external data before storing it; do not assume optional fields are present.
- **Duplicate spots:** Avoid creating duplicate spot records during manual entry or future API synchronization.

18.10 Database and API Reliability

- **Database unavailable:** Return a controlled server error and log the failure without exposing database credentials or internal stack traces.
- **Transaction failure:** Roll back related database changes when an operation that must be atomic fails.
- **Invalid foreign key:** Reject references to nonexistent users, activities, spots, or groups.
- **Concurrent requests:** Use database constraints and appropriate transaction handling to protect

data integrity.

- **Unexpected server exception:** Return a consistent error response and log diagnostic details securely.
- **Malformed request body:** Return HTTP 400 with a clear validation message.
- **Resource not found:** Return HTTP 404 when a requested resource does not exist or is not accessible under the defined API behavior.
- **Conflicting operation:** Return HTTP 409 when a request conflicts with an existing resource or business rule.
- **Network timeout:** Allow safe retries for operations where retrying will not create duplicate records.
- **CORS failure:** Configure allowed frontend origins correctly and do not weaken production security as a workaround.
- **Sensitive error information:** Never expose stack traces, JWT secrets, password hashes, database credentials, or other sensitive internal details to the client.

18.11 Feedback and Administration

- **Invalid rating:** Accept only ratings from 1 to 5.
- **Missing required feedback:** Validate required fields and allow optional comments to remain empty.
- **Unauthorized feedback access:** Enforce the defined permissions for viewing or modifying feedback.
- **Duplicate feedback:** Define whether users may submit multiple feedback records for one activity or update an existing record. Enforce that rule consistently.
- **Non-admin access:** Reject attempts to access administrative endpoints without the required role.
- **Admin deletes referenced data:** Prevent deletion from creating orphaned records; use defined deletion or deactivation rules.
- **Invalid analytics data:** Handle empty datasets and zero denominators without errors.
- **Sensitive user information:** Restrict administrative data access to authorized roles and avoid exposing password hashes or authentication tokens.

18.12 Security and Data Integrity

- Validate all input on the backend even when frontend validation exists.
- Enforce unique constraints for email, invite codes, memberships, swipes, and saved activities where specified.
- Never trust user IDs, roles, group membership, or ownership values supplied by the client without authorization checks.
- Prevent users from modifying another user's preferences, swipes, saved activities, or profile.
- Protect against mass assignment by accepting only explicitly permitted DTO fields.
- Use transactions for operations that require multiple related database changes to succeed together.
- Log relevant errors without logging passwords, JWTs, or other secrets.
- Return consistent error responses for expected failures and use HTTP status codes appropriate to the situation.

18.13 General User Experience

- Show loading indicators while requests are in progress.
- Show clear success messages when actions complete.
- Show actionable error messages when actions fail.
- Provide retry options for recoverable network failures.
- Display friendly empty states when no data is available.

- Prevent repeated submissions when a request is already processing.
- Preserve user input when practical after a recoverable failure.
- Never indicate that an operation succeeded until the server confirms success.
- Ensure that failure in a nonessential feature does not unnecessarily break unrelated application functionality.

Implementation requirement: These edge cases must be considered during implementation and testing. Prioritize authentication and authorization, data integrity, duplicate requests, group matching correctness, recommendation fallbacks, and graceful error handling. Not every edge case requires a separate feature, but every applicable case must have defined and testable behavior.

19. Definition of Done

- Database schema implemented and documented.
- Spring Boot application starts successfully with environment-based configuration.
- Authentication and JWT authorization work end to end.
- Activities and dummy spots can be managed through APIs.
- Multiple activity and spot images can be returned through APIs.
- Preferences are stored and used by recommendation/filter logic.
- Swipe and saved-activity flows work.
- Expandable activity card works in collapsed and expanded states.
- Horizontal and vertical gestures do not conflict.
- Groups can be created and joined.
- Group members can swipe and matches are calculated correctly.
- Plans and feedback can be created and retrieved.
- Admin endpoints are protected.
- Swagger documents the APIs.
- API responses use DTOs rather than directly exposing JPA entities.
- Frontend can consume the deployed backend APIs.
- Application is tested locally.
- Application is deployed successfully.

20. Success Criteria

- A new user can register, log in, complete preferences, discover activities, and swipe.
- A user can view a clean, image-focused activity card without excessive information.
- A user can expand an activity card to view detailed information and additional images.
- A user can like/reject activities using gestures or buttons.
- A user can save an activity and view it later.
- A user can create a group and invite others using a code.

- Multiple group members can swipe on the same activity set.
- The system correctly identifies unanimous and high-percentage group matches.
- A group can turn a match into a simple hangout plan.
- Admin can maintain the activity/spot/category catalog.
- The application remains usable with manually seeded data.
- The application does not depend on an external Places API during the initial implementation.
- Recommendation logic can be changed without major changes to unrelated modules.

21. Product Vision

HanginOut should evolve from a simple student hangout discovery website into a personalized social discovery platform.

The long-term system can combine user preferences, group behavior, location, activity metadata, weather, and machine learning to help people decide what to do together while keeping affordability as a core product principle.

The core product philosophy should remain:

Discover more. Spend less. Make memories together.

End of Document